
Especificação do Simulador: Arquitetura Computacional e Sistemas Operacionais

Fase 1: Processamento em Lote e Arquitetura RISC (16-bits)

Prof. Michel Pires da Silva

1 Introdução

A compreensão profunda da interação entre os componentes de *hardware* e as camadas fundamentais de *software* é um pilar indispensável para o domínio da computação moderna. Para consolidar essa visão sistêmica, o presente trabalho propõe o desenvolvimento da etapa fundamental de um simulador computacional. Nesta primeira versão, o sistema operará estritamente sob o paradigma de **processamento em lote** (*batch*). Nesse modelo inicial, análogo aos primeiros sistemas computacionais da história, as tarefas (*jobs*) são submetidas à memória e executadas de forma puramente sequencial. O simulador funcionará inicialmente como uma máquina *bare-metal*, isto é, sem a intervenção de escalonadores de processos, gerenciadores de memória avançados ou suporte a múltiplas CPUs ou sistemas de cache.

O núcleo de processamento do simulador modelará uma arquitetura hipotética fundamentada na filosofia **RISC** (*Reduced Instruction Set Computer*), operando com **palavras limitadas a 16 bits**. Essa restrição de tamanho não é acidental, possuindo um forte propósito pedagógico: ela exige da equipe uma modelagem matemática e estrutural rigorosa. Ao lidar com um espaço reduzido, os desenvolvedores são desafiados a projetar um conjunto de instruções eficiente, equilibrando a quantidade de *bits* destinados ao código da operação (*opcode*) e aos operandos (registradores ou endereços de memória). Ademais, a adoção da filosofia RISC impõe o paradigma *Load/Store*, no qual as operações lógicas e aritméticas ocorrem exclusivamente no âmbito dos registradores, separando claramente as etapas de computação e acesso à memória.

Por se tratar de um projeto prático e contínuo, é importante que o código-fonte seja concebido sob os princípios da **alta modularidade** e do **encapsulamento**. As decisões de engenharia de *software* tomadas nesta fase ditarão a viabilidade das próximas etapas da disciplina. Esta infraestrutura inicial constitui a base que, em trabalhos futuros, sofrerá modificações tanto na vertente física quanto lógica, a saber:

- **Evolução Arquitetural (*Hardware*):** O ciclo de instrução sequencial evoluirá para um modelo de execução sobreposta (*Pipeline*), introduzindo desafios de *hazards* de controle e dados. Paralelamente, o acesso direto à RAM será intermediado por uma hierarquia de memória mais veloz (*Cache*).
- **Evolução Sistêmica (Sistema Operacional):** O modelo em lote será substituído pela multiprogramação. Modelos de **escalonamento de processos** entrarão em ação para decidir a ordem e a fatia de tempo (*quantum*) em que as tarefas devem utilizar o processador, buscando mitigar a ociosidade gerada por gargalos de Entrada e Saída (I/O). Por fim, o endereçamento absoluto dará lugar à memória virtual, exigindo a implementação da tradução de endereços por meio da **Paginação**.

Dessa forma, o desenvolvimento inicial deste simulador não representa apenas um exercício de programação, mas a construção passo a passo de um ecossistema computacional completo.

2 Organização da Arquitetura

A concepção de um simulador computacional fiel à realidade exige que as restrições físicas do *hardware* sejam traduzidas para o *software* através de fronteiras arquiteturais rígidas. Em outras palavras, as estruturas de dados e as funções do código devem respeitar o isolamento e as vias de comunicação (*barramentos*) que existem em uma placa-mãe real. O sistema é composto por módulos estritamente encapsulados. A Figura 1 ilustra o fluxo de execução, as vias de comunicação e a topologia de separação dos componentes que devem ser implementados.

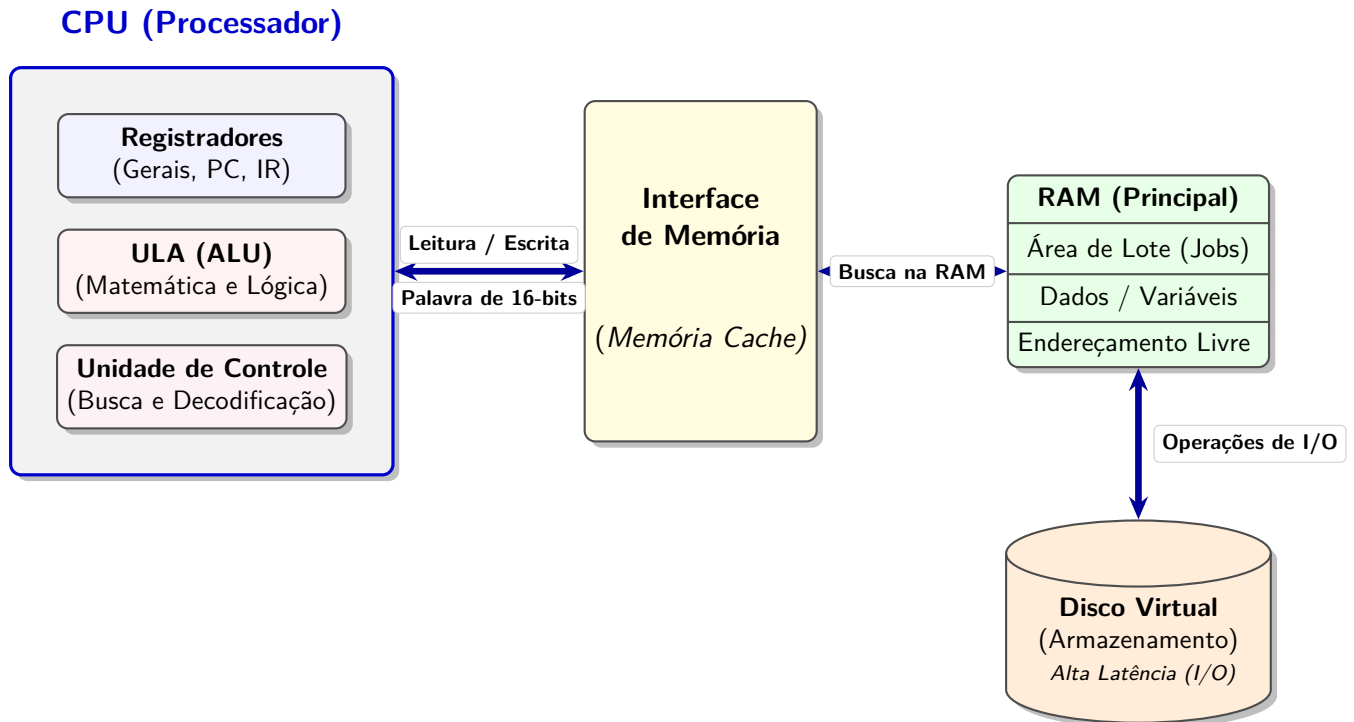


Figure 1: Arquitetura modular do simulador. O isolamento dos componentes prepara o sistema para expansões futuras, como Cache, Escalonamento e Paginação.

Para a implementação computacional desta primeira etapa, a sala deve estruturar o código refletindo os seguintes subsistemas:

- **Unidade Central de Processamento (CPU):** O cérebro do simulador. Deve ser dividida internamente em três frentes:
 - **Registradores:** Memória de acesso imediato que compõe o estado e o contexto do processador. Inclui os registradores de uso geral (onde os cálculos efetivamente ocorrem) e os de propósito específico, como o *Program Counter* (PC), que aponta para a próxima instrução, e o *Instruction Register* (IR), que armazena a instrução que está sendo decodificada no momento.
 - **Unidade Lógica e Aritmética (ULA):** O componente puramente calculista, responsável pelas operações matemáticas (soma, subtração) e operações *booleanas* (AND, OR). É vital garantir que a ULA não possua ponteiros diretos para a memória externa.
 - **Unidade de Controle (UC):** O "maestro" da CPU. É a lógica (função ou classe) responsável por buscar a instrução (atualizando o PC), mascarar seus *bits* para descobrir o *opcode*, e coordenar a ULA e os Registradores para executar a ação correspondente.

- **Interface de Memória:** Como evidenciado na Figura 1, a CPU não acessa a RAM diretamente. Essa decisão de projeto (uso de uma função mediadora) garante que, no futuro, a inserção de uma memória Cache ocorra de forma transparente para a CPU, evitando a reescrita maciça de código.
- **Memória Principal (RAM) e Secundária (Disco):** O armazenamento primário abrigará as instruções e variáveis das tarefas em lote (*jobs*). Já a memória secundária servirá para simular requisições de Entrada e Saída (I/O) que, devido à sua natureza eletromecânica simulada, imporão penalidades altíssimas ao relógio (*clock*) do sistema.

O alinhamento correto das responsabilidades desses módulos é o que garante o cumprimento do princípio *Load/Store* exigido pela arquitetura RISC, detalhado a seguir.

Dica: O Paradigma Load/Store

Em arquiteturas RISC, instruções matemáticas como ADD e SUB **nunca** buscam dados na memória. Elas operam **apenas** sobre registradores. Toda transferência de dados deve ocorrer unicamente através das instruções de interface: LOAD (traz o dado da Memória → Registrador) e STORE (salva o resultado do Registrador → Memória).

2.1 Conjunto Mínimo de Instruções (Assembly)

Para que o processador simulado seja capaz de executar algoritmos reais, é necessário definir a sua interface de *software*, ou seja, o seu **Conjunto de Instruções** (*Assembly*). Sendo uma arquitetura de 16 bits, o espaço é restrito, exigindo escolhas precisas. A sala tem liberdade para definir os códigos numéricos (*opcodes*) exatos e a quantidade de registradores, mas o simulador **deve obrigatoriamente** implementar o conjunto mínimo de instruções listado na Tabela 1, garantindo a capacidade de realizar cálculos, acessar a memória e controlar o fluxo de execução (desvios lógicos).

Categoria	Mnemônico	Descrição e Ação Esperada
Transferência (Memória)	LOAD	Busca um dado da memória e o armazena em um registrador.
	STORE	Copia o valor atual de um registrador e o salva na memória.
ULA (Matemática e Lógica)	ADD	Soma os valores de dois registradores e salva no registrador de destino.
	SUB	Subtrai o valor de dois registradores.
	AND	Operação lógica bit a bit (Conjunção).
	OR	Operação lógica bit a bit (Disjunção).
Controle de Fluxo	BEQ	<i>Branch if Equal</i> : Compara dois registradores; se forem iguais, o <i>Program Counter</i> (PC) salta para o endereço fornecido.
	JUMP	Salto incondicional. O PC é alterado para o endereço informado imediatamente.
Sistema	HALT	Sinaliza o término do programa atual, instruindo o simulador a carregar a próxima tarefa do lote.

Table 1: Conjunto de instruções obrigatório para a arquitetura RISC simulada.

Composição de Tarefas (Exemplo de Job)

Como os algoritmos rodarão em *bare-metal* (sem Sistema Operacional), a equipe deverá escrever pequenos trechos de código em formato numérico (ou criar um montador simples) para testar a máquina.

Por exemplo, para realizar a operação $C = A + B$ (onde A, B e C estão na RAM), o programa **não pode** fazer um ADD direto na memória. O fluxo deve ser obrigatoriamente:

1. LOAD R1, EnderecoA (Traz A para CPU)
2. LOAD R2, EnderecoB (Traz B para CPU)
3. ADD R3, R1, R2 (Soma dentro da ULA)
4. STORE R3, EnderecoC (Salva o resultado na RAM)
5. HALT (Finaliza a tarefa)

Exemplo: Ciclo de Decodificação e Execução (Instrução Tipo I)

Entrada (Memória/IR): 0x124F

Decodificação (Mascaramento em C/C++):

```
1 uint16_t IR = 0x124F; // Instrucao lida da RAM
2 uint16_t opcode = (IR >> 12) & 0x000F; // Isola os 4 bits iniciais
3 uint16_t regDest = (IR >> 8) & 0x000F; // Isola os 4 bits do registrador
4 uint16_t ender = IR & 0x00FF; // Isola os 8 bits finais (
    Endereco)
```

Interpretação:

- opcode = 0x1 (Supondo que seja a operação LOAD)
- regDest = 0x2 (Registrador R2)
- ender = 0x4F (Posição 79 da RAM)

Processamento (Execução da UC comandando a Máquina):

```
1 if (opcode == 1) { // Logica do LOAD (sem acento)
2     uint16_t dado = LerMemoria(ender);
3     Registradores[regDest] = dado;
4 }
5 PC = PC + 1; // Avanca contador para a proxima palavra
```

Saída (Estado do Simulador): O registrador R2 agora possui o valor que estava armazenado no endereço 0x4F da memória, o PC foi incrementado e a variável clockGlobal foi acrescida pela função LerMemoria.

3 Hierarquia de Memória e Custos (Clock)

3.1 Interface da Memória Principal

A CPU **não deve** possuir acesso direto ao vetor (*array*) que representa a RAM. Toda leitura/escrita deve passar por uma função mediadora.

```
1 uint16_t RAM[65536]; // Representacao da memoria fisica (16-bits)
2 extern int clockGlobal;
3
4 // Interface mediadora
5 uint16_t LerMemoria(uint16_t endereco) {
```

```

6     clockGlobal++; // Acesso a memoria consome 1 ciclo
7     return RAM[endereco];
8 }

```

Listing 1: Isolamento do barramento de memória

Motivação: No Trabalho 2, a equipe não precisará alterar a CPU. Bastará modificar a função `LerMemoria` para que ela verifique primeiramente a **Cache**.

3.2 Memória Secundária (Disco) e Simulação de I/O

Tarefas reais executam I/O e pausam o uso da CPU. Para que os futuros algoritmos de escalonamento saibam diferenciar tarefas *CPU-bound* de *I/O-bound*, simule a lentidão do disco mecânico forçando uma busca sequencial e aplicando severas penalidades ao *clock*.

Modelo de Custo: I/O de Disco

Entrada: Requisição de busca do dado 0x4A2B no Disco.

Processamento:

```

1 int LerDisco(uint16_t dadoBuscado) {
2     for (int i = 0; i < TAM_DISCO; i++) {
3         clockGlobal += 50; // Penalidade: 50 clocks por setor percorrido
4         if (Disco[i] == dadoBuscado) return i; // Retorna o indice
5     }
6     return -1;
7 }

```

Saída: A função pode retornar após varrer 100 índices, adicionando **5000 ciclos de clock** ao relógio global da simulação.

4 Exemplo Completo: Ciclo de Execução em Lote

Para consolidar o entendimento do fluxo de dados e controle da máquina, esta seção apresenta o ciclo de vida completo de uma tarefa (*job*) simples. O objetivo do programa é calcular $C = A + B$.

A arquitetura operará em modo de lote (*batch*): o simulador lê as tarefas do Disco, carrega a primeira na Memória RAM, executa instrução por instrução até encontrar o comando **HALT**, e então limpa o contexto (zera PC e registradores temporários) para carregar o próximo *job*.

4.1 Passo 1: O Job no Disco (Memória Secundária)

No Disco, o programa não passa de uma sequência de números inteiros de 16 *bits* (representados aqui em hexadecimal para facilitar a leitura). Assumiremos o seguinte mapeamento de *opcodes*: 1 = LOAD, 2 = ADD, 3 = STORE, F = HALT.

Disco Virtual (Matriz ou Arquivo txt)

Programa 1 (Soma de A e B):

- 0x100A → LOAD R0, [0x0A] (Carrega o valor do endereço 10 para R0)
- 0x110B → LOAD R1, [0x0B] (Carrega o valor do endereço 11 para R1)
- 0x2201 → ADD R2, R0, R1 (Soma R0 e R1, salva em R2 - Formato Tipo R)
- 0x320C → STORE R2, [0x0C] (Salva R2 no endereço 12)
- 0xF000 → HALT (Fim da tarefa)

4.2 Passo 2: Carregamento (Loader)

Antes da execução iniciar, o simulador deve varrer o disco, extrair o *job* atual e seus dados iniciais, e copiá-los para a RAM.

```
1 // O simulador carrega as instrucoes no inicio da RAM (Endereco 0)
2 RAM[0] = 0x100A; RAM[1] = 0x110B; RAM[2] = 0x2201;
3 RAM[3] = 0x320C; RAM[4] = 0xF000;
4
5 // O simulador carrega os dados (Variaveis A e B) nos enderecos especificados
6 RAM[10] = 5; // Variavel A = 5
7 RAM[11] = 7; // Variavel B = 7
8 RAM[12] = 0; // Variavel C (Vazia inicialmente)
```

Listing 2: Cópia do Job do Disco para a RAM

4.3 Passo 3: Execução (Ciclo de Máquina)

Com a RAM populada, o *Program Counter* (PC) é zerado (PC = 0) e o laço principal (*loop*) da Unidade de Controle é iniciado.

Ciclo 1: PC = 0

- **Busca:** IR = LerMemoria(0); (Traz 0x100A, clock++)
- **Decodificação:** Opcode = 1 (LOAD), Destino = R0, Endereço = 10.
- **Execução:** R0 = LerMemoria(10); (R0 recebe 5, clock++)
- **Atualização:** PC++ (PC agora é 1)

Ciclo 2: PC = 1

- **Busca:** IR = LerMemoria(1); (Traz 0x110B, clock++)
- **Decodificação:** Opcode = 1 (LOAD), Destino = R1, Endereço = 11.
- **Execução:** R1 = LerMemoria(11); (R1 recebe 7, clock++)
- **Atualização:** PC++ (PC agora é 2)

Ciclo 3: PC = 2

- **Busca:** IR = LerMemoria(2); (Traz 0x2201, clock++)
- **Decodificação:** Opcode = 2 (ADD), R_Dest = 2, R_Fonte1 = 0, R_Fonte2 = 1.
- **Execução:** R2 = R0 + R1; (R2 recebe 5 + 7 = 12) *Nota: Sem acesso a RAM, clock inalterado pela execucao.*
- **Atualização:** PC++ (PC agora é 3)

Ciclo 4: PC = 3

- **Busca:** IR = LerMemoria(3); (Traz 0x320C, clock++)
- **Decodificação:** Opcode = 3 (STORE), Origem = R2, Endereço = 12.
- **Execução:** EscreverMemoria(12, R2); (RAM[12] recebe 12, clock++)
- **Atualização:** PC++ (PC agora é 4)

Ciclo 5: PC = 4

- **Busca:** IR = LerMemoria(4); (Traz 0xF000, clock++)
- **Decodificação:** Opcode = F (HALT).
- **Execução:** Sinaliza o fim do *job* atual. O simulador encerra este laço, exibe as métricas de tempo (clocks consumidos) e carrega o próximo *job* do lote, reiniciando o PC.

5 Conclusão e Entregáveis

O desenvolvimento deste *core* é a primeira etapa para compreender a complexidade sistêmica de um computador. Ao exigir o encapsulamento de módulos e o controle rigoroso do relógio, o projeto prepara o alicerce lógico para que, nas próximas etapas da disciplina, a equipe implemente o Sistema Operacional (Escalonador de Processos e Gerência de Memória/Páginas) e refine a Arquitetura (Pipeline e Cache) de maneira totalmente orgânica.

O simulador deverá receber como entrada um lote de tarefas (arquivos `.txt` ou `.bin` contendo as instruções numéricas geradas pela equipe). Ao final da execução de cada programa do lote (quando a UC encontrar um `HALT`), o sistema deverá imprimir na tela um **Relatório de Execução**. Esse mesmo relatório deve também compor um arquivo `output.dat` para fins de análise futura por cada equipe.

Modelo Esperado de Saída (Terminal do Simulador)

```
=====
[SIMULADOR RISC 16-BITS] - EXECUCAO EM LOTE
=====
Carregando Job 1:  soma_vetor.txt...
Execucao finalizada (HALT encontrado no PC 0x001A).

-- ESTADO DA CPU --
PC: 0x001B          IR: 0xF000
Registradores Gerais:
R0: 0x0005   R1: 0x0007   R2: 0x000C   R3: 0x0000
R4: 0x0000   R5: 0x0000   R6: 0x0000   R7: 0x0000

-- ESTATISTICAS DO JOB --
Instrucoes Executadas:    14
Acessos a Memoria (RAM):  6
Acessos ao Disco (I/O):   1
CLOCK TOTAL DESTE JOB:    5120 ciclos
=====
Iniciando Job 2...
```

Checklist de Entrega

O trabalho deverá ser entregue através do repositório versionado, contendo obrigatoriamente:

1. **Código-fonte do Simulador:** Escrito em C/C++, devidamente comentado e estruturado de forma modular (respeitando as fronteiras arquiteturais).
2. **Documentação da Arquitetura (Manual do *Assembly*):** Um documento (PDF ou Markdown) descrevendo os formatos de instrução escolhidos pela equipe, a quantidade de registradores e a tabela de *Opcodes*.
3. **Cargas de Teste (*Jobs*):** Pelo menos 2 (dois) arquivos de lote contendo programas distintos e funcionais escritos no *Assembly* projetado pela equipe, prontos para serem executados pelo simulador.

Equipe: Única (Trabalho em grupo conforme definição em sala)

Data limite de entrega: 04 de outubro de 2026

Formato de entrega: Link do repositório Git (via plataforma AVA).